UD3 – Integración de herramientas de computación en la nube con aplicaciones

Índice

1. Introducción **1**
3.1 Conexión entre la aplicación y el servicio de IA **1**
3.2 Preparación de los datos de entrada **4**
3.3 Transformación de los datos de salida **5**
3.4 Diseño de la interfaz de usuario **6**
3.5 Documentación y control de errores **8**
4. Caso práctico integrador **10**
5. Glosario **11**
6. Recursos y referencias **11**

1. Introducción

Las aplicaciones modernas rara vez implementan sus propios modelos de inteligencia artificial desde cero. En su lugar, **consumen servicios de IA en la nube** (APIs) ofrecidos por proveedores como Azure AI Services, AWS AI/ML, Google Cloud AI, Anthropic o OpenAI. Esto permite a un desarrollador añadir capacidades como visión artificial, procesamiento de lenguaje natural, generación de texto o reconocimiento de voz sin necesidad de entrenar ni alojar modelos propios.

Integrar estos servicios implica mucho más que "hacer una llamada a una API". Un desarrollador profesional debe:

- Establecer una **conexión segura y autenticada** con el servicio.
- **Preparar y validar** los datos que se envían al modelo.
- **Interpretar y transformar** la respuesta para que sea útil dentro de la aplicación.
- Ofrecer una **interfaz de usuario** clara y usable para interactuar con la IA.
- **Documentar** el proceso y **controlar los errores** que puedan surgir en cualquier punto del flujo.

Esta unidad didáctica se organiza en torno a los cinco criterios de evaluación del RA3: conexión con el servicio, preparación de la entrada, transformación de la salida, diseño de la interfaz y documentación con control de errores.

> **Dos escenarios posibles de servicio de IA**: los ejemplos de esta unidad incluyen proveedores externos (Anthropic, OpenAI, Azure AI, AWS AI…) y un modelo propio expuesto mediante una API (por ejemplo, un modelo de scikit-learn servido con FastAPI o Flask). En ambos casos hay que proteger el servicio. El mecanismo de autenticación depende de la arquitectura, la exposición y los requisitos de seguridad: una API propia puede necesitar autenticación y autorización, especialmente si es accesible desde una red compartida o desde Internet. El apartado **3.1.5** desarrolla ese caso.

3.1 Conexión entre la aplicación y el servicio de IA

3.1.1 Arquitectura general

La mayoría de los servicios de IA en la nube se exponen mediante **APIs REST** sobre HTTPS. El flujo típico es:

```
Aplicación cliente  --->  Petición HTTP (JSON)  --->  Servicio de IA en la nube
Aplicación cliente  <---  Respuesta HTTP (JSON)  <---  Servicio de IA en la nube
```

Componentes clave que hay que identificar antes de programar la integración:

- **Endpoint**: la URL a la que se envían las peticiones (p. ej. `https://api.miproveedor.com/v1/messages`).
- **Método HTTP**: normalmente `POST` para enviar datos a analizar/generar, `GET` para consultar estado o listar recursos.

- **Región y versión de la API**: en proveedores externos (Azure, AWS, GCP) suele haber que fijar una región y una versión de API concretas; en una API propia (ver 3.1.5) esto no aplica.

3.1.2 Autenticación y autorización (servicios de terceros)

Un servicio de IA externo (Anthropic, OpenAI, Azure AI, AWS AI…) siempre exige **credenciales**. Los mecanismos más habituales son:

- **API key**: clave secreta que el proveedor espera en una cabecera o en otro mecanismo documentado. No existe una cabecera universal: por ejemplo, OpenAI utiliza habitualmente `Authorization: Bearer ...`, mientras que Anthropic utiliza `x-api-key` y una cabecera de versión. Hay que consultar la documentación del servicio concreto.
- **OAuth 2.0 / token Bearer**: se obtiene un token de acceso y se envía, por ejemplo, mediante `Authorization: Bearer ...`.
- **Claves de suscripción y roles IAM**: mecanismos específicos de algunos proveedores, como Azure y AWS, que permiten controlar el acceso a recursos.

Autenticación comprueba quién o qué solicita acceso; **autorización** determina qué operaciones tiene permitido realizar. No son conceptos equivalentes.

> ⚠ **Buena práctica de seguridad**: las claves y los tokens secretos no deben incrustarse en el código ni subirse a un repositorio público. En desarrollo pueden cargarse desde variables de entorno y un archivo `.env` excluido mediante `.gitignore`; en producción, es preferible usar un gestor de secretos. Las credenciales deben tener los permisos mínimos necesarios y poder rotarse o revocarse.

3.1.3 SDK vs llamadas HTTP directas

	Llamadas HTTP directas	SDK oficial del proveedor
Ventajas	No añade dependencias; control total sobre la petición	Gestiona autenticación, reintentos y serialización automáticamente
Inconvenientes	Hay que construir manualmente cabeceras, cuerpo y gestión de errores	Añade una dependencia externa; menos control de bajo nivel
Cuándo usarlo	Prototipos rápidos, o cuando se llama a una API propia sencilla (ver 3.1.5)	Proyectos en producción contra un proveedor externo con SDK disponible

Ejemplo conceptual de llamada HTTP directa con `fetch` (JavaScript, ejecutado en el servidor):

```js
async function llamarServicioIA(prompt) {
  const apiKey = process.env.IA_API_KEY; // Variable disponible en el entorno del servidor
  if (!apiKey) throw new Error("Falta configurar IA_API_KEY");

  const respuesta = await fetch("https://api.proveedor.com/v1/generate", {
    method: "POST",
    headers: {
      "Content-Type": "application/json",
      "Authorization": `Bearer ${apiKey}` // La cabecera depende del proveedor
    },
    body: JSON.stringify({ prompt, max_tokens: 300 }),
    signal: AbortSignal.timeout(15000)
  });
  if (!respuesta.ok) throw new Error(`Error HTTP ${respuesta.status}`);
  return await respuesta.json();
}
```

La URL, las cabeceras y los nombres de los campos son ilustrativos y deben adaptarse a la API elegida. Este código debe ejecutarse en un backend (por ejemplo, Node.js), no en el navegador: las variables privadas del servidor no están disponibles automáticamente en el cliente y una clave incluida en JavaScript enviado al navegador puede quedar expuesta. Una arquitectura habitual es **navegador → backend propio → servicio de IA**.

3.1.4 Buenas prácticas al establecer la conexión

- Comprobar el **código de estado HTTP** (2xx = éxito; 4xx = error del cliente; 5xx = error del servidor).
- Configurar un **timeout** razonable para evitar que la aplicación se quede bloqueada esperando respuesta.
- Implementar **reintentos con backoff exponencial** ante errores temporales.
- Usar **HTTPS** siempre, nunca HTTP sin cifrar.

3.1.5 Caso particular: conexión con una API propia

Cuando el "servicio de IA" es **un modelo propio desplegado por el equipo** (por ejemplo, el modelo entrenado en la Práctica 1 y desplegado en la Práctica 2), el endpoint y el control de acceso los define el propio proyecto.

- En un entorno local aislado puede ser suficiente no añadir autenticación externa para una práctica didáctica. Sin embargo, si la API se comparte en una red o se publica en Internet, debe evaluarse e implementar la autenticación y autorización que correspondan. Ser una API propia no la hace segura por sí sola.
- El **endpoint** es una URL propia (por ejemplo, `POST /predict` en el servicio desplegado en la Práctica 2), en lugar de una URL de un proveedor externo.
- La **validación de los datos de entrada** (apartado 3.2) pasa a hacerse en el propio servidor, normalmente con la ayuda de un framework como **FastAPI** (usando modelos `pydantic`, que validan tipos y campos obligatorios de forma automática) o **Flask** (con validación manual o con librerías como `marshmallow`).
- Se debe seguir un formato de respuesta **consistente y predecible**, con una estructura fija para el éxito y otra para el error, de modo que la interfaz siempre sepa qué esperar. Por ejemplo, estas son dos respuestas alternativas (cada bloque es JSON válido):

  Respuesta de éxito:

  ```
  { "prediccion_usd": 245000.0 }
  ```

  Respuesta de error:

  ```
  { "error": "Falta el campo MedInc" }
  ```

- Los **códigos de estado HTTP** siguen aplicando: `200` para una predicción correcta, `422` normalmente para errores de validación automática de los modelos de entrada de FastAPI/Pydantic, `400` para otros errores de petición que la aplicación decida tratar así y `500` para errores internos no controlados.

Ejemplo didáctico con FastAPI y `pydantic` (validación automática; no incluye autenticación):

```
from fastapi import FastAPI
from pydantic import BaseModel
import joblib
import numpy as np

app = FastAPI()
modelo = joblib.load("modelo_california.pkl")

class DatosVivienda(BaseModel):
    MedInc: float
    HouseAge: float
    AveRooms: float
    AveBedrms: float
    Population: float
    AveOccup: float
    Latitude: float
    Longitude: float

@app.post("/predict")
def predecir(datos: DatosVivienda):
    entrada = np.array([[datos.MedInc, datos.HouseAge, datos.AveRooms,
                         datos.AveBedrms, datos.Population, datos.AveOccup,
                         datos.Latitude, datos.Longitude]])
    prediccion = modelo.predict(entrada)[0]
    # Multiplicar por 100000 solo si el objetivo se entrenó en la escala original
    # de California Housing y no se transformó durante el entrenamiento.
    return {"prediccion_usd": round(float(prediccion) * 100000, 2)}
```

Si falta un campo obligatorio o el valor no cumple el tipo declarado, FastAPI devuelve normalmente un error `422 Unprocessable Entity` con detalles de validación, sin necesidad de escribir esa comprobación a mano. No es lo mismo que un `400`, que la aplicación puede utilizar para otros errores de petición. En Flask, la validación equivalente debe implementarse o añadirse mediante una biblioteca.

> **Nota de seguridad:** este ejemplo se centra en la validación y no implementa autenticación. No debe publicarse tal cual como servicio abierto en Internet. Una API accesible desde Internet debe protegerse según su contexto: autenticación, autorización, HTTPS, límites de peticiones y de tamaño, y registro de actividad sin almacenar secretos. Los errores que se envían al cliente deben ser informativos pero no revelar trazas, rutas internas ni credenciales.

3.2 Preparación de los datos de entrada

Antes de enviar información a un servicio de IA, los datos deben adaptarse al **formato, tamaño y estructura** que ese servicio espera. Un error muy común del alumnado es asumir que cualquier dato "tal cual" puede enviarse al modelo.

3.2.1 Tipos de datos de entrada según el servicio

Tipo de servicio de IA	Formato típico de entrada
Procesamiento de lenguaje natural (NLP)	Texto plano o JSON con campos `prompt`, `messages`, `documents`
Visión artificial	Imagen codificada en Base64 o URL pública de la imagen
Reconocimiento de voz (Speech-to-Text)	Audio binario (WAV, MP3) o *stream* de audio
Traducción automática	Texto plano + código de idioma origen/destino (ISO 639-1)
Análisis de sentimiento / clasificación	Texto plano o lote (batch) de textos en JSON
Regresión/clasificación sobre datos tabulares (modelo propio de scikit-learn, por ejemplo)	JSON con campos numéricos nombrados, uno por cada variable del modelo (por ejemplo, `{"MedInc": 8.3, "HouseAge": 41, ...}`)

3.2.2 Fases de la preparación de datos

1. **Recolección**: obtener el dato desde su origen (formulario, base de datos, fichero subido, cámara, micrófono…).
2. **Limpieza y normalización**:
   - Eliminar caracteres de control, espacios redundantes o HTML no deseado.
   - Normalizar codificación de caracteres a **UTF-8**.
   - Convertir a minúsculas solo cuando el caso de uso lo requiera. No es una limpieza universal: puede alterar nombres propios, identificadores o textos donde las mayúsculas tengan significado.
3. **Validación**:
   - Comprobar que el campo obligatorio no esté vacío.
   - Verificar el **tamaño máximo** admitido (por ejemplo, límite de tokens en un modelo de lenguaje, o tamaño máximo de imagen en MB).
   - Comprobar el **tipo MIME** declarado (`image/png`, `audio/wav`…), sin confiar exclusivamente en él: cuando se aceptan archivos subidos, conviene comprobar también su contenido, limitar su tamaño y rechazar formatos no admitidos.

   Transformación al formato requerido: por ejemplo, convertir una imagen a **Base64** antes de enviarla, o empaquetar un texto en la estructura JSON exigida por la API:

4. ```
   { "messages": [{ "role": "user", "content": "Clasifica el sentimiento de esta reseña: ..." }],
    ↪  "max_tokens": 100 }
   ```

5. **Segmentación (chunking)**: cuando el texto de entrada supera el límite de tokens del modelo, se divide en fragmentos más pequeños que se procesan por separado y luego se combinan los resultados.

3.2.3 Consideraciones importantes

- **Privacidad y protección de datos (RGPD)**: antes de enviar datos personales a un servicio de IA externo, hay que verificar si el proveedor los trata conforme al RGPD, dónde se almacenan y si se usan para reentrenar modelos. Cuando sea posible, se deben **anonimizar o seudonimizar** los datos sensibles antes de enviarlos.
- **Coste**: muchos servicios cobran por volumen de datos enviados (tokens, minutos de audio, número de imágenes), por lo que preparar bien los datos también implica optimizar su tamaño.
- **Codificación de parámetros**: parámetros como `temperature`, `max_tokens`, `idioma` o `modelo` forman parte de los "datos de entrada" en un sentido amplio, y deben configurarse según el caso de uso.

3.2.4 Caso particular: validación de datos tabulares (modelo de regresión propio)

Cuando el servicio de IA es un modelo de regresión o clasificación entrenado sobre datos tabulares (como el modelo de precios de vivienda del proyecto integrador), la validación de entrada de la sección 3.2.2 se concreta así:
   ```

- **Comprobar que estén presentes todos los campos obligatorios**: el modelo espera un número fijo de variables (por ejemplo, las 8 variables del dataset *California Housing*); si falta alguna, la API debe devolver un error de validación controlado (422 normalmente en FastAPI/Pydantic, o el código que implemente la aplicación en Flask).
- **Comprobar el tipo de cada campo**: todos deben ser numéricos; un valor de texto donde se espera un número debe provocar un error de validación controlado.
- No suele haber, en este caso, fases de "limpieza de texto" o "conversión a Base64" (propias de NLP o visión); la preparación se reduce a validar presencia, tipo y, opcionalmente, rango razonable de cada variable.

```python
# Ejemplo de validación manual (Flask), equivalente a lo que hace
# automáticamente un modelo pydantic en FastAPI (ver apartado 3.1.5)
import numpy as np

CAMPOS_REQUERIDOS = ["MedInc", "HouseAge", "AveRooms", "AveBedrms",
                     "Population", "AveOccup", "Latitude", "Longitude"]

def validar_datos(datos_json):
    faltantes = [c for c in CAMPOS_REQUERIDOS if c not in datos_json]
    if faltantes:
        return False, f"Faltan los campos: {', '.join(faltantes)}"
    for campo in CAMPOS_REQUERIDOS:
        valor = datos_json[campo]
        # En Python, bool es una subclase de int: se excluye explícitamente.
        if isinstance(valor, bool) or not isinstance(valor, (int, float)):
            return False, f"El campo '{campo}' debe ser numérico"
        if not np.isfinite(valor):
            return False, f"El campo '{campo}' debe ser un número finito"
    return True, None
```

3.3 Transformación de los datos de salida

La respuesta que devuelve un servicio de IA rara vez se puede mostrar "tal cual" al usuario final. Hay que **extraer, interpretar y adaptar** la información recibida a las estructuras de datos y a la interfaz de la aplicación.

3.3.1 Estructura habitual de una respuesta

La mayoría de los servicios devuelven un **JSON** con metadatos adicionales, no solo el resultado:

```
{ "id": "resp_12345", "resultado": { "etiquetas": [{ "nombre": "gato", "confianza": 0.97 }] }, "uso": {
 ↪  "tokens_salida": 12 } }
```

3.3.2 Tareas típicas de transformación

1. **Parsing (análisis sintáctico)**: convertir el JSON de texto en objetos manipulables del lenguaje de programación.

   ```javascript
   const datos = await respuesta.json();
   const etiquetaPrincipal = datos.resultado.etiquetas[0].nombre;
   ```

2. **Filtrado**: descartar resultados por debajo de un umbral de confianza.

   ```python
   etiquetas_validas = [
       e for e in datos["resultado"]["etiquetas"] if e["confianza"] > 0.8
   ]
   ```

3. **Mapeo (mapping)**: convertir la estructura del proveedor a un **modelo de datos propio** de la aplicación, para no acoplar el resto del sistema al formato específico de un proveedor (esto facilita cambiar de proveedor en el futuro).

   ```python
   def mapear_a_modelo_interno(respuesta_ia):
       return Etiqueta(
           texto=respuesta_ia["nombre"],
           porcentaje=round(respuesta_ia["confianza"] * 100, 1)
       )
   ```

4. **Enriquecimiento y formateo**: combinar el resultado con datos propios de la aplicación y adaptarlo a la interfaz (redondear cifras, dar formato de fecha, traducir etiquetas).
5. **Persistencia**: guardar el resultado transformado en base de datos si la aplicación necesita histórico o trazabilidad.

3.3.3 Validación de la respuesta recibida

Antes de utilizar una respuesta, la aplicación debe comprobar que tiene la estructura esperada. Una respuesta HTTP satisfactoria no garantiza que el contenido sea válido: podría faltar una clave, llegar una lista vacía o recibirse un tipo de dato inesperado.

```
def extraer_etiqueta_principal(datos):
    try:
        etiquetas = datos["resultado"]["etiquetas"]
        if not etiquetas:
            return None
        return etiquetas[0]["nombre"]
    except (KeyError, TypeError, IndexError):
        return None  # La aplicación debe gestionar la respuesta incompleta
```

En una aplicación real, conviene registrar el fallo sin guardar datos personales innecesarios y mostrar un mensaje claro. No se debe asumir que una respuesta externa siempre contiene todos los campos esperados.

3.3.4 Caso particular: formatear una predicción numérica (regresión)

Cuando el modelo devuelve un único valor numérico (por ejemplo, un precio estimado) en lugar de una lista de etiquetas con confianza, la transformación de salida es más sencilla, pero sigue aplicando el mismo principio: **el dato "en bruto" no se muestra directamente al usuario, se adapta a un formato legible**.

```
# La variable objetivo del dataset California Housing está en
# cientos de miles de dólares; se transforma a un precio legible
def formatear_prediccion(valor_bruto: float) -> dict:
    precio_usd = round(valor_bruto * 100_000, 2)
    return {"prediccion_usd": precio_usd}
```

```
>>> formatear_prediccion(2.45)
{'prediccion_usd': 245000.0}
```

Aquí no hay ni *parsing* de un JSON externo (el modelo devuelve directamente un número de Python) ni filtrado por confianza, pero sí hay **formateo para presentación** (pasar de "2.45" a "245000.0 USD"), que es la tarea de transformación que realmente pide este caso de uso.

3.3.5 Respuestas en streaming (caso no aplicable a APIs de predicción simples)

Algunos servicios de generación de texto devuelven la respuesta en fragmentos progresivos (*streaming*) en lugar de esperar al resultado completo. No es el caso de una API de predicción como `/predict` (que devuelve un único valor de golpe), pero conviene conocer el concepto si en el futuro se integra un servicio de generación de texto.

3.4 Diseño de la interfaz de usuario

Una integración técnicamente correcta puede fracasar si el usuario final no entiende cómo interactuar con la funcionalidad de IA. El diseño de la interfaz debe tener en cuenta las particularidades de los sistemas de IA: **latencia, incertidumbre y posibilidad de error**.

3.4.1 Principios de diseño UX para funciones de IA

- **Feedback de estado (loading states)**: las respuestas de IA pueden tardar segundos; mostrar un indicador de carga (*spinner*, barra de progreso) para que el usuario sepa que se está procesando.
- **Transparencia sobre la IA**: indicar claramente cuándo un contenido ha sido generado o analizado por IA.
- **Gestión de la incertidumbre**: si el resultado incluye un nivel de confianza, mostrarlo o usarlo para advertir al usuario.
- **Posibilidad de corrección**: permitir que el usuario edite, reintente o rechace el resultado propuesto por la IA.

3.4.2 Componentes de interfaz habituales

Componente	Uso típico
Formulario de entrada (campos numéricos, texto, chat)	Recoger los datos que necesita el modelo
Indicador de carga / *spinner*	Mostrar que la IA está procesando la solicitud
Panel de resultado	Mostrar la predicción o respuesta ya formateada
Notificaciones de error	Informar de fallos de conexión o datos inválidos

3.4.3 Ejemplo simplificado de flujo de interfaz (HTML + JS, opción Flask/Jinja2)

```html
<form id="formulario-ia">
  <textarea id="entrada" placeholder="Escribe tu consulta..."></textarea>
  <button type="submit">Enviar</button>
</form>
<div id="estado" aria-live="polite"></div>
<div id="resultado"></div>
```

```javascript
const formulario = document.getElementById("formulario-ia");
const entrada = document.getElementById("entrada");
const estado = document.getElementById("estado");
const resultadoDiv = document.getElementById("resultado");

formulario.addEventListener("submit", async (e) => {
  e.preventDefault();
  estado.textContent = "Procesando con IA...";
  resultadoDiv.textContent = "";
  try {
    const respuestaIA = await llamarServicioIA(entrada.value);
    resultadoDiv.textContent = transformarResultado(respuestaIA);
    estado.textContent = "";
  } catch (error) {
    estado.textContent = "Ha ocurrido un error. Inténtalo de nuevo.";
  }
});
```

El ejemplo presupone que `llamarServicioIA` y `transformarResultado` están definidas en el código de la aplicación. Se muestran los tres pilares conectados: conexión, transformación de salida y control de errores. La clave del proveedor debe permanecer en el backend; no debe incluirse en el JavaScript que se entrega al navegador.

3.4.4 Ejemplo con Streamlit (interfaz en Python puro)

Para proyectos en los que el resto del stack ya está en Python (como el modelo y la API), **Streamlit** o **Gradio** permiten construir la interfaz sin salir de Python y sin escribir HTML/JS, aplicando los mismos principios de diseño del apartado 3.4.1 (indicador de carga, gestión de errores, posibilidad de corrección):

```
import streamlit as st
import requests

st.title("Estimador de precio de vivienda")

ingreso_medio = st.number_input("Ingreso medio (decenas de miles $)", value=5.0)
habitaciones = st.slider("Número medio de habitaciones", 1.0, 15.0, 5.0)
poblacion = st.number_input("Población de la zona", value=1000.0)

if st.button("Calcular precio estimado"):
    with st.spinner("Consultando el modelo..."):  # feedback de estado
        payload = {
            "MedInc": ingreso_medio, "HouseAge": 30, "AveRooms": habitaciones,
            "AveBedrms": 1.0, "Population": poblacion, "AveOccup": 3.0,
            "Latitude": 34.0, "Longitude": -118.0,
        }
        try:
            respuesta = requests.post("http://localhost:8000/predict", json=payload, timeout=5)
            respuesta.raise_for_status()
            datos_salida = respuesta.json()
            precio = datos_salida.get("prediccion_usd")
            if not isinstance(precio, (int, float)) or isinstance(precio, bool):
                raise ValueError("La respuesta no contiene una predicción numérica válida")
            st.success(f"Precio estimado: {precio:,.0f} USD")
        except requests.exceptions.RequestException:
            st.error("No se ha podido contactar con el servicio de predicción. Inténtalo de nuevo.")
        except (ValueError, TypeError):
            st.error("El servicio ha devuelto una respuesta no válida.")
```

Nótese que aplica los cinco pilares del RA3: **conexión** (`requests.post`), **datos de entrada** (`payload` con los 8 campos), **datos de salida** (lectura y validación de `prediccion_usd`), **interfaz** (`st.spinner`, `st.success`) y **control de errores** (`try/except` con `st.error`). Los valores fijos de las variables que no pide el formulario son solo demostrativos; en una aplicación real se deben solicitar, justificar como valores predeterminados o derivar de datos válidos. Además, la escala y el significado de cada variable deben coincidir con los utilizados al entrenar el modelo.

3.5 Documentación y control de errores

3.5.1 Documentación de los procesos

Documentar la integración con un servicio de IA es fundamental para el mantenimiento futuro del proyecto y para que otros desarrolladores (o el propio alumno, meses después) entiendan cómo funciona. Debe incluir como mínimo:

- **Descripción general** del servicio de IA utilizado y su propósito dentro de la aplicación.
- **Requisitos de configuración**: variables de entorno o URL del servicio necesarias.
- **Formato de los datos de entrada y salida**, con ejemplos de peticiones y respuestas.
- **Requisitos de acceso y seguridad**: autenticación, permisos, gestión de secretos y uso de HTTPS cuando corresponda.
- **Límites conocidos**: campos obligatorios, tamaños máximos, coste si aplica y comportamiento ante errores.

Un ejemplo de estructura de documentación técnica (por ejemplo, en un `README.md` del proyecto):

```
## Integración con el servicio de IA

### Descripción
Este módulo llama al endpoint /predict para estimar el precio de una vivienda.

### Configuración necesaria
- URL del servicio: https://mi-servicio.example.com (usar la URL real del despliegue y HTTPS en producción).

### Flujo de datos
1. El usuario introduce los datos en el formulario.
2. La aplicación valida y envía el JSON al endpoint /predict.
3. El servicio devuelve la predicción en formato `{"prediccion_usd": ...}`.
4. La aplicación muestra el precio formateado al usuario.

### Límites conocidos
- Los 8 campos numéricos son obligatorios.
```

3.5.2 Control de errores

Una integración robusta debe anticipar los distintos tipos de fallo que pueden producirse en cualquier punto del proceso:

Tipo de error	Causa habitual	Estrategia de control
Error de red / timeout	Sin conexión, servicio caído, latencia alta	Reintentos con backoff exponencial, mensaje claro al usuario
Autenticación (401/403) / cuota (429)	Solo aplica a servicios externos con credenciales o límites de plan	Verificar credenciales o esperar y reintentar (ver nota siguiente para APIs propias)
Datos de entrada inválidos (400/422)	Formato incorrecto o campo obligatorio ausente	Validar antes de enviar y documentar el código que devuelve la API (ver 3.2.4)
Error interno del servicio (500/503)	Fallo en el modelo o en el servidor	Registrar el incidente y mostrar un mensaje claro, sin exponer la traza técnica

> **Caso de una API propia** (ver 3.1.5): los errores 401/403/429 también pueden aparecer si la API implementa autenticación, autorización o límites de uso. En FastAPI, la validación automática de modelos de entrada suele devolver `422`; la aplicación puede responder `400` para otros errores de petición y `500` ante fallos internos no controlados.

Ejemplo de manejo de errores con reintentos (Python):

```python
import time
import logging
import requests

logger = logging.getLogger("integracion_ia")

def llamar_con_reintentos(funcion_peticion, intentos=3):
    """Reintenta timeouts y errores HTTP transitorios; no reintenta todo error."""
    if intentos < 1:
        raise ValueError("intentos debe ser al menos 1")

    for intento in range(1, intentos + 1):
        try:
            return funcion_peticion()
        except (requests.Timeout, TimeoutError) as error:
            logger.warning("Timeout en intento %s/%s", intento, intentos)
            if intento == intentos:
                raise
        except requests.HTTPError as error:
            codigo = error.response.status_code if error.response is not None else None
            # Solo se reintentan algunos errores temporales del servidor o de límite.
            if codigo not in (429, 500, 502, 503, 504) or intento == intentos:
                raise
            logger.warning("Error HTTP temporal %s en intento %s/%s", codigo, intento, intentos)
        except requests.RequestException:
            # Otros errores de petición no se reintentan automáticamente aquí.
            raise

        if intento < intentos:
            time.sleep(min(2 ** (intento - 1), 8))  # backoff exponencial limitado

    raise RuntimeError("Se agotaron los reintentos con el servicio de IA")
```

Este ejemplo requiere que `funcion_peticion` use la biblioteca `requests`. No deben reintentarse indiscriminadamente todos los errores: un `401` por credenciales incorrectas o un `400/422` por datos inválidos no se arreglan esperando. Además, los reintentos deben reservarse para operaciones que puedan repetirse de forma segura; algunas operaciones pueden generar costes o efectos duplicados.

Buenas prácticas de control de errores:

- Diferenciar entre **errores recuperables** (reintentar) y **no recuperables** (informar y detener).
- **Nunca** mostrar al usuario final detalles técnicos sensibles (claves, trazas internas).
- Registrar (*logging*) los errores con contexto suficiente para poder depurarlos después.
- Definir un **comportamiento de reserva (fallback)**: por ejemplo, si el servicio no responde, mostrar un mensaje alternativo en vez de bloquear la aplicación.

Pruebas básicas de integración

Antes de dar por terminada la integración, conviene comprobar al menos estos casos:

Prueba	Resultado esperado
Petición válida	Respuesta correcta y datos de salida con la estructura documentada.
Campo obligatorio ausente o tipo incorrecto	Error de validación controlado (`400` o `422`, según la API).
Servicio lento o no disponible	Timeout o error controlado; la interfaz informa al usuario.
Respuesta sin la clave esperada o con tipo incorrecto	La aplicación detecta la respuesta inválida y no falla mostrando un error sin contexto.
Credenciales inválidas o permisos insuficientes (si se usa autenticación)	Respuesta `401`/`403` y registro del problema sin exponer secretos.
Límite de uso alcanzado (si existe)	Tratamiento del `429` conforme a la política del proveedor.

Estas pruebas se pueden realizar manualmente con la interfaz o mediante herramientas como `pytest` y un cliente de pruebas del framework.

4. Caso práctico integrador

Escenario: una aplicación de gestión de incidencias incorpora un asistente de IA (un servicio externo) que clasifica cada incidencia (urgente/normal/baja) y genera un resumen. Aplicando los cinco criterios del RA3: **(a)** se autentica con una API Key guardada en variable de entorno; **(b)** el texto se limpia y se valida su

longitud antes de enviarlo; **(c)** la respuesta JSON se parsea y se mapea a un objeto `Incidencia` propio; **(d)** se muestra un indicador de carga y la prioridad con un color distintivo, con opción de corrección manual; **(e)** se documenta el flujo en el README y se controlan timeouts, cuota excedida y respuestas mal formadas.

El apartado 4.1 desarrolla el mismo recorrido para el caso concreto de este módulo: una API propia de predicción, inicialmente presentada en un entorno de prácticas.

4.1 Caso aplicado: API propia de predicción de precios de vivienda

El mismo recorrido, aplicado a una **API propia de predicción** (ver 3.1.5), usando el modelo de regresión entrenado sobre el dataset *California Housing*:

1. **Conexión**: se expone un endpoint `POST /predict` en la API desplegada (Práctica 2), que recibe y devuelve JSON. En un entorno local aislado de prácticas puede llamarse sin credenciales; si se publica o comparte el servicio, se debe valorar e implementar autenticación, autorización, HTTPS y límites de uso. En un despliegue real no se debe asumir que una URL pública puede quedar abierta sin protección.
2. **Datos de entrada**: se valida (con `pydantic` en FastAPI o manualmente en Flask, ver 3.2.4) que la petición incluya las 8 variables numéricas del modelo. FastAPI/Pydantic devuelve normalmente `422` cuando falla la validación automática; Flask devolverá el código que implemente la aplicación.
3. **Datos de salida**: si el modelo se entrenó con el objetivo original `MedHouseVal` de California Housing y no se transformó la variable objetivo durante el entrenamiento, el valor predicho se expresa en unidades de 100.000 dólares aproximadamente; se multiplica por 100.000 para presentarlo en dólares (ver 3.3.4). Esta transformación debe coincidir con el preprocesamiento y la escala usados al entrenar el modelo. La respuesta mantiene una estructura fija: `{"prediccion_usd": 245000.0}`.
4. **Interfaz de usuario**: una interfaz en Streamlit (ver 3.4.4) pide al usuario el ingreso medio, el número de habitaciones y la población, usa valores por defecto para el resto de variables, muestra un indicador de carga mientras espera la respuesta y presenta el precio estimado.
5. **Documentación y errores**: el `README.md` documenta el endpoint `/predict`, sus 8 campos de entrada, el método de autenticación si procede y ejemplos de petición/respuesta. La API registra los fallos internos y devuelve un código adecuado sin exponer la traza técnica al usuario final. Se prueban tanto respuestas correctas como errores de validación, timeouts y caídas del servicio.

5. Glosario

- **API (Application Programming Interface)**: conjunto de reglas que permite que una aplicación se comunique con otro sistema o servicio.
- **Backoff exponencial**: estrategia de reintento en la que el tiempo de espera entre intentos aumenta exponencialmente.
- **Autenticación**: proceso de comprobar la identidad de quien solicita acceso.
- **Autorización**: determinación de los recursos y operaciones a los que una identidad puede acceder.
- **Endpoint**: URL concreta a la que se dirige una petición a una API.
- **HTTPS**: versión de HTTP protegida mediante cifrado TLS durante el transporte.
- **JSON (JavaScript Object Notation)**: formato ligero de intercambio de datos, ampliamente usado por las APIs de IA.
- **SDK (Software Development Kit)**: conjunto de herramientas y librerías que facilita el uso de una API concreta.
- **Token**: unidad de texto que procesan los modelos de lenguaje; también se usa para referirse a credenciales de autenticación (*token de acceso*), según el contexto.
- **Timeout**: límite de tiempo de espera tras el cual una petición se cancela o se considera fallida.

6. Recursos y referencias

- Documentación de FastAPI: `https://fastapi.tiangolo.com/`
- Documentación de Streamlit: `https://docs.streamlit.io/`
- Documentación de la API de Anthropic (Claude), como referencia de un servicio externo real: `https://docs.claude.com/`
- Documentación de la API de OpenAI: `https://platform.openai.com/docs/`
- Guía de buenas prácticas RGPD y tratamiento de datos con terceros: sitio oficial de la Agencia Española de Protección de Datos (AEPD).